

Laboratório BFS DFS e UCS

Resultados esperados e explicação das cinco experiências

Este guia apresenta as respostas ao notebook `IA_BFS_DFS_UCS_laboratorio.ipynb`. Os valores foram obtidos executando o código e os parâmetros pedidos. A correção deve considerar os resultados e a explicação das decisões, distinguindo caminho da solução, custo e trabalho de procura.

Antes de interpretar os resultados

O grafo é dirigido. A pesquisa começa em S e tem G1, G2 e G3 como objetivos terminais. A ordem de geração é alfabética, exceto quando se escolhe a ordem inversa. A posição dos nós no desenho não define a ordem dos sucessores.

OPEN contém estados que aguardam processamento. Nas tabelas, o próximo a retirar aparece à esquerda. Cada entrada mostra estado e custo acumulado g; apenas UCS utiliza g como prioridade. VISITED inclui todos os estados retirados, incluindo objetivos.

O teste de objetivo ocorre na retirada de OPEN, antes da expansão. Retirar um objetivo aumenta a contagem de retiradas, mas não a de expansões. Os três objetivos não têm sucessores neste laboratório.

As regras dos três algoritmos

BFS usa uma fila FIFO. DFS usa uma pilha LIFO e empilha pela ordem inversa à exploração pretendida. Ambos ignoram estados já presentes em OPEN ou VISITED e conservam o primeiro predecessor.

UCS retira o menor custo acumulado, usando FIFO em empate. Atualiza custo e predecessor perante uma melhoria estrita em OPEN, sem alterar a antiguidade da entrada. Com os custos positivos deste laboratório, não é necessário reabrir estados retirados.

Condições dos resultados

Experiência	Configuração
1	BFS, DFS e UCS; parar no primeiro objetivo.
2	BFS e DFS; sucessores por ordem alfabética e inversa.
3	Custos A→G1 de 9, 1 e 30; restantes custos fixos.
4	TODOS=False e True; custos originais e ordem alfabética.
5	Novo grafo; atualizar OPEN desativado e ativado.

Os resultados são determinísticos para estas regras. Alterar a ordem de geração, a gestão dos empates ou o tratamento dos repetidos pode mudar ordens de retirada e caminhos.

Experiência 1 Fila pilha e prioridade

Objetivo e condições

Comparar os três algoritmos no grafo original, por ordem alfabética, terminando ao retirar o primeiro objetivo. O custo $A \rightarrow G1$ é 9.

Resultados esperados

Algoritmo	Objetivo	Caminho	Custo	Prof.
BFS	G1	$S \rightarrow A \rightarrow G1$	14	2
DFS	G1	$S \rightarrow A \rightarrow G1$	14	2
UCS	G2	$S \rightarrow D \rightarrow C \rightarrow G2$	13	3

Algoritmo	Ordem de retirada	Retiradas	Expansões
BFS	S, A, B, D, G1	5	4
DFS	S, A, G1	3	2
UCS	S, A, D, B, C, E, G2	7	6

Como explicar a escolha seguinte

Depois de S, BFS apresenta [A:5, B:9, D:6] e retira A por ser a frente da fila. Depois de A, a fila é [B:9, D:6, G1:14]: o objetivo ainda espera pelos estados anteriores.

DFS também retira A primeiro. Ao expandir A, B já está em OPEN e não é empilhado de novo. G1 é o novo topo, pelo que sai a seguir. Por isso, esta execução retira S, A, G1, sem passar por B.

UCS retira A com custo 5 e melhora B de 9 para 8, através de A. A fronteira passa a [D:6, B:8, G1:14]. D sai antes de B e descobre C e E com custo 8. B sai antes de C e E por ter entrado primeiro. O objetivo de menor custo é G2, com $6 + 2 + 5 = 13$.

Exemplos de estados repetidos

Em BFS e DFS, $A \rightarrow B$ é ignorado porque B já está em OPEN. Em BFS, $B \rightarrow A$ é ignorado porque A já está em VISITED. Na curta execução DFS desta experiência ainda não ocorre uma rejeição por VISITED antes de parar em G1; o aluno pode indicá-lo, em vez de inventar um evento. Em UCS, $D \rightarrow S$ é um exemplo de rejeição por VISITED.

Resposta esperada

A ordem de retirada descreve o trabalho de procura; o caminho resulta dos predecessores do objetivo. BFS retira B e D, mas o seu caminho é $S \rightarrow A \rightarrow G1$. DFS encontra rapidamente esse mesmo objetivo neste caso, sem garantir menor custo ou profundidade em geral. UCS encontra G2 com custo 13, inferior a 14.

Experiência 2 Mudar a ordem dos sucessores

Objetivo

Observar o efeito da ordem alfabética e da ordem inversa em BFS e DFS, mantendo os mesmos custos e a paragem no primeiro objetivo.

Resultados esperados

Ordem	Algoritmo	Objetivo	Custo	Prof.
Alfabética	BFS	G1	14	2
Alfabética	DFS	G1	14	2
Inversa	BFS	G1	14	2
Inversa	DFS	G3	15	3

Alfabética, BFS: S, A, B, D, G1.

Alfabética, DFS: S, A, G1.

Inversa, BFS: S, D, B, A, E, C, G1.

Inversa, DFS: S, D, E, G3.

O que muda em BFS

A ordem inversa começa por colocar D, B e A na fila. A ordem de retirada muda para S, D, B, A, E, C, G1, mas a solução continua a ser $S \rightarrow A \rightarrow G1$, com profundidade 2 e custo 14. Os objetivos G2 e G3 estão a uma profundidade maior.

BFS mantém a garantia de profundidade mínima. Quando existem várias soluções à mesma profundidade mínima, a ordem pode decidir qual é devolvida. Essa garantia não significa custo mínimo quando as ligações têm custos diferentes.

O que muda em DFS

Com ordem alfabética, empilham-se inicialmente D, B e A: A fica no topo. Com ordem inversa, a exploração pretendida é D, B, A, pelo que se empilham A, B e D. D fica no topo e a pesquisa segue para E e depois G3.

A solução DFS inversa é $S \rightarrow D \rightarrow E \rightarrow G3$, com custo $6 + 2 + 7 = 15$ e profundidade 3. A DFS alfabética devolve $S \rightarrow A \rightarrow G1$, com custo 14 e profundidade 2. Estados já em OPEN não são reposicionados: é necessário considerar esta regra ao interpretar a pilha.

Resposta esperada

A ordem dos sucessores influencia o trabalho de pesquisa e pode mudar a solução. BFS continua a procurar por níveis, enquanto DFS segue o ramo disponível no topo da pilha. A menor profundidade corresponde ao menor número de ligações; o menor custo corresponde à menor soma dos respetivos pesos. São critérios distintos.

Experiência 3 Alterar os custos das ligações

Objetivo e condições

Alterar apenas o custo $A \rightarrow G1$ para 9, 1 e 30. A estrutura do grafo, a ordem alfabética e as restantes ligações mantêm-se. Cada algoritmo termina ao retirar o primeiro objetivo.

Resultados esperados

Custo $A \rightarrow G1$	BFS objetivo / custo	DFS objetivo / custo	UCS objetivo / custo
9	G1 / 14	G1 / 14	G2 / 13
1	G1 / 6	G1 / 6	G1 / 6
30	G1 / 35	G1 / 35	G2 / 13

Custos dos melhores caminhos

O caminho para G1 é $S \rightarrow A \rightarrow G1$ e custa $5 + \text{CUSTO_AG1}$. Assim, passa a custar 14, 6 ou 35. Os caminhos ótimos para G2 e G3 mantêm os custos 13 e 15, respetivamente.

BFS e DFS continuam a devolver $S \rightarrow A \rightarrow G1$. Os pesos mudam o custo calculado, mas não a escolha dos estados: BFS decide pela fila e DFS pela pilha. A profundidade da solução mantém-se em 2.

Como interpretar UCS

Com custo 9, UCS escolhe G2 por 13, antes de G1 por 14. Com custo 1, G1 custa apenas 6 e passa a ser a melhor solução. Com custo 30, G1 custa 35 e UCS continua a escolher G2 por 13.

No caso $\text{CUSTO_AG1}=1$, depois de expandir A, D e G1 têm custo 6. D sai primeiro porque já estava em OPEN. Esse desempate não muda o resultado: G1 é retirado depois, ainda com o menor custo entre os objetivos.

Resposta esperada

Só UCS utiliza o custo acumulado para ordenar a fronteira. BFS conserva a solução de profundidade mínima, mas pode devolver um caminho muito mais caro: com $A \rightarrow G1=30$, devolve custo 35 quando existe G2 de custo 13. A DFS encontra a mesma solução que BFS nestes três testes, mas isso é uma consequência da ordem e deste grafo, não uma garantia do algoritmo.

Conclusão

Alterar pesos pode mudar a solução de UCS sem mudar a solução de BFS ou DFS. Se o objetivo é minimizar distância, energia ou outra soma de custos positivos, a seleção por custo acumulado responde a esse critério; contar ligações é um critério diferente.

Experiência 4 Continuar até aos três objetivos

Objetivo e condições

Com TODOS=False, obtêm-se os resultados da Experiência 1. Com TODOS=True, preservam-se OPEN, VISITED, custos e predecessores e continua-se até retirar os três objetivos terminais, com custos originais e ordem alfabética.

Ordem e custos esperados

Algoritmo	Ordem dos objetivos	Custos na mesma ordem
BFS	G1 → G2 → G3	14, 15, 15
DFS	G1 → G3 → G2	14, 25, 15
UCS	G2 → G1 → G3	13, 14, 15

Caminhos devolvidos

Algoritmo	Objetivo	Caminho	Prof.
BFS	G1	S → A → G1	2
BFS	G2	S → B → C → G2	3
BFS	G3	S → D → E → G3	3
DFS	G1	S → A → G1	2
DFS	G3	S → B → C → F → G3	4
DFS	G2	S → B → C → G2	3
UCS	G2	S → D → C → G2	3
UCS	G1	S → A → G1	2
UCS	G3	S → D → E → G3	3

Como interpretar as diferenças

BFS devolve profundidades 2, 3 e 3. C conserva B como pai por ter sido descoberto por B. G2 custa 15, apesar de existir um caminho por D de custo 13.

Na DFS, depois de G1, a pesquisa segue por B e C. F precede G2 na ordem alfabética e conduz a G3, com custo 25. A pesquisa termina com D ainda em OPEN.

UCS garante custos não decrescentes: 13, 14 e 15. BFS também apresenta custos não decrescentes neste exemplo, mas isso é coincidência; ordena por profundidade.

Continuar reutiliza OPEN, VISITED e os predecessores. Reiniciar para cada objetivo cria pesquisas independentes a partir de S e repete trabalho; não reproduz a sequência de uma única execução contínua.

Experiência 5 Atualizar um estado em OPEN

Objetivo e grafo usado

O início é S e o único objetivo é G. Os arcos são $S \rightarrow A$ (4), $S \rightarrow B$ (1), $B \rightarrow A$ (1), $A \rightarrow G$ (2) e $B \rightarrow G$ (10). Com `ATUALIZAR_OPEN=False`, a versão ignora melhorias e é deliberadamente incompleta. `True` aplica a atualização correta de UCS.

Resultados esperados

Atualizar OPEN	Caminho	Custo
False	$S \rightarrow B \rightarrow G$	11
True	$S \rightarrow B \rightarrow A \rightarrow G$	4

Fronteira com atualizações corretas

Operação	OPEN após a operação
Inicial	[S:0]
Expandir S	[B:1, A:4]
Expandir B	[A:2, G:11]
Expandir A	[G:4]
Retirar G	[]

Ao expandir B, o caminho para A custa $1 + 1 = 2$, melhor do que 4. Atualizam-se $g(A)=2$ e $\text{pai}(A)=B$. Na mesma expansão, G é descoberto com custo $1 + 10 = 11$ e $\text{pai}(G)=B$.

Ao expandir A com o custo corrigido, encontra-se $g(G)=2 + 2 = 4$. Atualizam-se o custo de G e o seu pai, que passa de B para A. O caminho final é $S \rightarrow B \rightarrow A \rightarrow G$, com custo 4.

O que acontece quando se ignoram melhorias

A conserva custo 4 e pai S. Quando A é expandido, o candidato para G custa $4 + 2 = 6$, mas também é ignorado porque G já está em OPEN. G conserva custo 11 e pai B. A solução devolvida é $S \rightarrow B \rightarrow G$, de custo 11, e não 6.

Nas duas versões, a ordem de retirada é S, B, A, G. Logo, a mesma ordem de retirada não garante o mesmo caminho nem o mesmo custo: os registos de custo e predecessor são essenciais.

Resposta esperada

Em UCS, um estado descoberto pode receber um caminho melhor antes de sair de OPEN. A regra de rejeitar todos os repetidos impede essa melhoria e destrói a garantia de custo ótimo. Neste exemplo não há reabertura de VISITED: A e G ainda aguardam processamento quando são atualizados.

Terminar logo ao descobrir G, durante a expansão de B, devolveria custo 11. Esperar pela retirada permite que A melhore esse valor para 4. Com custos positivos e atualizações corretas, é na retirada que UCS confirma o menor custo do objetivo.